

Breaking the Ice for Agile Development of Embedded Software: An Industry Experience Report

Peter Manhart¹ and Kurt Schneider^{1,2}

¹DaimlerChrysler Research Center, Ulm, Germany

²now: FG Software Engineering, Universität Hannover, Germany

Peter.Manhart@DaimlerChrysler.com

Kurt.Schneider@Inf.Uni-Hannover.de

Abstract

A software engineering department in a DaimlerChrysler business unit was highly professional at developing embedded software for busses and coaches. However, customer specific add-ons were a regular source of hassle. Simple as they are, those individual requirements have to be implemented in hours or days rather than weeks or months. Poor quality or late upload into the bus hardware would cause serious cost and overhead. Established software engineering methods were considered inadequate and needed to be cut short.

Agile methods offer guidance when quality, flexibility and high speed need to be reconciled. However, we did not adopt any full agile method, but added single agile practices to our “process improvement toolbox”. We suggested a number of classical process improvement activities (such as more systematic documentation and measurement) and combined them with agile elements (e.g. Test First Process). This combination seemed to foster acceptance of agile ideas and may help us to break the ice for a cautious extension of agile process improvement.

1. Introduction

The Software Process Department at DaimlerChrysler Research and Technology supports all business units. We have a more than ten-year experience in software process improvement: We help to establish measurement programs [1], carry out CMM [2] or related [3] improvements. In particular, we have applied SPICE (ISO 15 504) and tailored it to our business unit needs. In many cases, we start from a goal-driven perspective [4] and apply experience-based process improvement [5], [6]. We consider all these approaches “classical software process improvement”.

About three years ago, we started to investigate agile approaches as a potential new source of ingredients for our improvement toolbox. While we found a number of IT-projects that had successfully adopted and applied agile methods, business units that produce software to be embedded into our vehicles seemed more reluctant to give agility a try.

During the last year, a bus and coach business unit asked us to support improving a particular software process variant. In premium-class busses and coaches, software plays a more and more important role. Although the business unit mastered software module development very well for series functionality, they felt more challenged by features that were added or modified by an individual customer – often late in the process.

We conducted a first overview analysis, then applied a goal-driven approach to derive process improvement suggestions. We recommended seven “classical” improvement activities – and two agile ones. After management had rejected two (classical) proposals, we started to implement the remaining seven activities. The well-known agile practices of “Unit Test” and “Test First” were included and treated just like the classical improvement suggestions.

We expected automated Unit Test to be a good door-opener: At first glance, the developers already created formal software specifications of the unit functionalities as a part of their documentation. So it seemed a quick win with few additional overhead to automate checking of the code against the specifications early. This would be a fundamental process change but still based on concepts that are familiar to most developers. Once accepted, it could be the first step towards Test First and to an even broader adoption of agile practices [7]. We prefer a smooth, step-to-step approach to revolutionary change of development practices. In a highly competitive business environment, the latter could be too risky and would not be accepted by the business units – since so little is known about agile method impacts.

2. Software embedded in premium busses

busses, both chances and specific risks of embedded software are of utmost importance.

2.1 Technology and software in busses

A modern bus contains between four and six programmable controller units ("FPS" in Figure 1). Controllers are connected by a low speed CAN and offer the following functions:

1. Gateway: They are communication gateways to fix programmed controllers at aggregate level, e.g. to the electronic brake control unit.
2. IO: They convert hardware signals to CAN Messages.
3. Bus Functions: On the basis of a real time operating system they compute and control bus-specific functionality like control of doors, climate, or audio/video equipment.

Customer-specific software functionality is always realized as bus functions, in our case taking the form of so-called “function plans (FUPs)” according to IEC1131-3. When a customer requests a special functionality, a decision is made, whether it will be fully or partially implemented in software. In that case, the engineering department has to produce one or more new function plans (FUPs) within time and quality constraints.

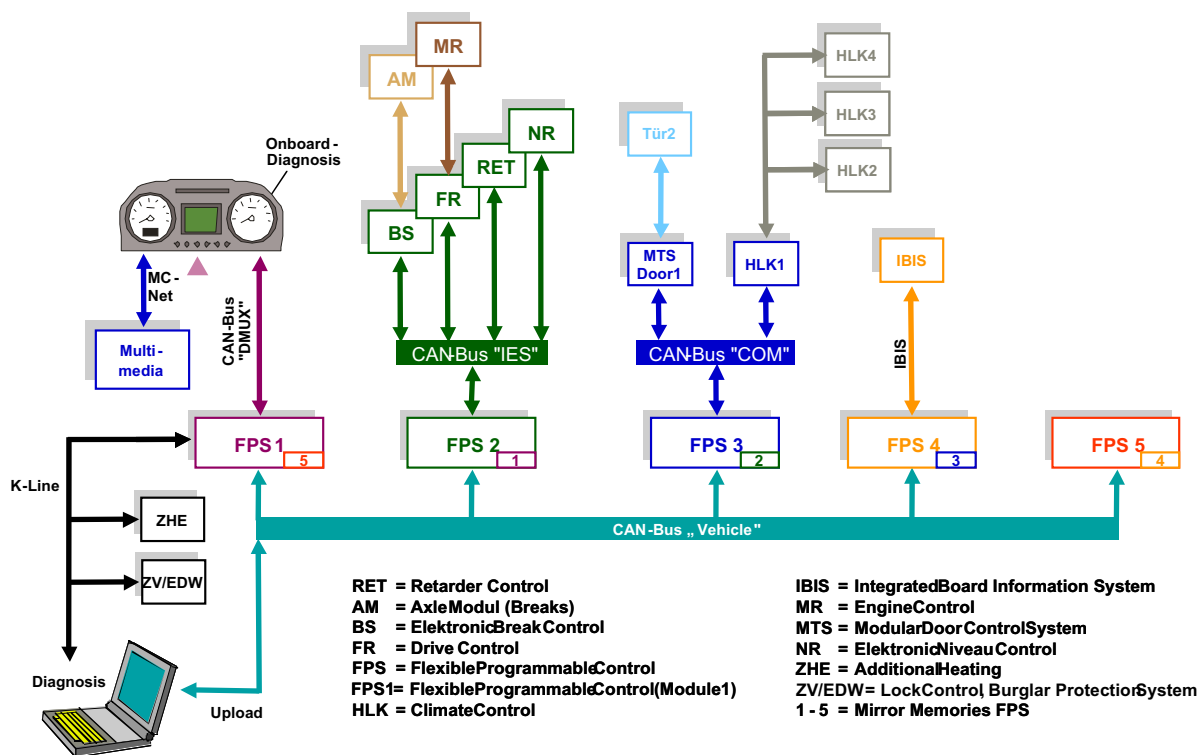


Figure 1: Hardware architecture of a modern bus. FPS is a programmable control unit.

2.2 Specifics of embedded bus&coach software

The conditions of engineering embedded software for high-end busses differ from those in many other automotive software engineering processes:

1. A higher degree of customer-specific functionality combined with greater time pressure and the necessity to deliver very high quality.
2. Customers buying busses do not buy off the shelf. Their own varying business models result in a high degree of individualization.
3. Resulting non-standard equipment, functions, or parameter sets are implemented by integrating different proportions of third party- and OEM-manufactured components.
4. Software for complex electric or electronic functions often runs on programmable computer controllers. In such a software environment, change *seems* easy.
5. Customers buying busses make a major investment. They demand top quality. When new requirements come to their mind, they expect them to be considered, and denial of late changes due to software process problems are rarely acceptable.

All of these issues increase time pressure for the realization of high quality customer-specific embedded software. These particularities also prevent any “existing” solutions from other car business units from being simply adopted. We needed to find custom-made solutions.

2.3 Developing software-based functions

The process for developing customer-specific functions is identical to regular (series) function

development, except for the higher time pressure and a more challenging requirements engineering task. Figure 2 shows the software process for software-based functions that we found in our initial analysis.

The engineering process starts when a *technical sales representative* decides to implement a customer specific function in software. Then he or she writes a function description and forwards it to the affected engineering department. When the software team receives an order it selects an engineer responsible for the desired functionality. The engineer schedules the software order according to the planned upload date. Sometimes ongoing work has to be interrupted to realize the function in time, sometimes the demanded date cannot be met at all.

The *software developer* then implements the FUP according to the requirements by using a graphical programming tool named LogiCAD, documents the FUP by using a database front-end, and specifies a plan for systematically checking the units functionality manually on a centralized hardware configuration. Since C code is generated by LogiCAD, a software developer has only limited opportunities to verify correctness: there is just a simulation feature which requires substantial manual effort.

After that, the *engineering team manager* checks FUPs and enclosed documentation and forwards them to the *system integration department*. They review the document for quality attributes and manually execute the FUP with according to its test plan. If review and execution results are positive, the function is released to *production* for upload to the bus. If any review or check fails (on either team or integration levels) the FUP is returned to the developer.

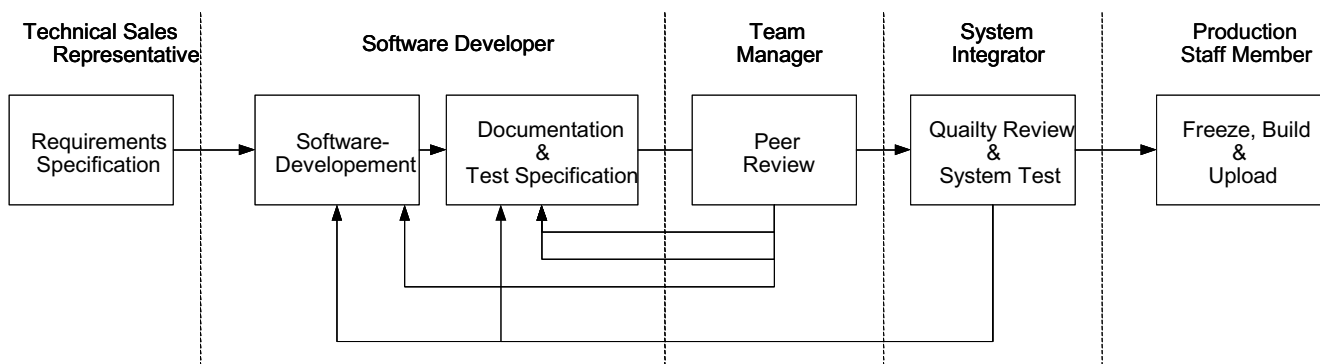


Figure 2: Process of software development, including quality assurance and upload to the bus

2.4 Timely delivery

When producing software for individual products like busses, software development is highly meshed with production. From the viewpoint of a production plant manager software is just a “part” like an engine. At a specific point of the assembly line the software part has to be ready for installation (“upload”) like any other part. A delay of any part causes severe economic consequences:

Without successfully completed upload, a bus cannot be driven. Even within the assembly hall, it needs to be towed around. This is costly in itself, and such a bus cannot be delivered, thus causing further cost.

Therefore, a software configuration will be uploaded regardless whether all customer-specific functions are ready or not. If not, a bus might first be uploaded without them. When those parts of the software have to be supplemented or exchanged later during production, delivery and aftersales processes get out of regular order. A manually coordinated process named “Special Upload” is processed instead. In addition to its higher cost, this process is less mature and bears higher risks. A high percentage of timely delivery of customer-specific functions is obviously an important goal.

2.5 Challenges in customer-specific functions

There are various reasons that can cause late delivery of a customer-specific software component:

1. **Unrealistic planning:** Development, quality assurance, integration and release cannot be shortened below a certain threshold. If a change request is received after the last possible date to complete the FUPs in time, a delay is unavoidable – no matter what.
2. **Expert unavailable:** Writing embedded software requires special domain knowledge, e.g. on engines or door physics. When all experts are engaged in “more important tasks”, software gets neglected and will be late at upload.
3. **Changing priorities:** priorities may be unclear or ignored. Changing the production plan has a major impact on the orders: if priorities and sequence of implementation of FUPs depends on their *delays*, an order may be completed in time that could not have been met under a first-in-first-out policy. Such adaptations, however, require a dynamic view of the production plan – which is difficult.
4. **Requirements misunderstood or changed:** A software developer or a sales representative with vague ideas about the desired functionality may cause several changes and iteration cycles to clarify requirements. Regular software components are usually well defined. Unfortunately, customer-

specific functions combine development pressure with rather unclear and changing requirements.

3. Related work:

High-speed software engineering

The desire to speed up software development is as old as the observation of vague or changing requirements. Literature gives a good recount of the various attempts to solve the problem:

- Boehm’s spiral model [8] emphasized the concept of risk-driven iterations. By addressing one risk at a time, the software system should be able to avoid overly lengthy breadth-first analysis phases. Since special-request FUP development is usually short (and often simple), there is no room for additional cycles.
- Prototyping is a well-known approach to handling unclear requirements. There are different variants that are tuned to elicit different aspects of a new system [9], [10]. For the above-cited reasons (rather small and low-complexity FUPs), any formal prototyping approach would be overkill. On the other hand, one could consider the current practice of developing special-request FUP code under pressure as “prototyping” – but, from that perspective, *product* development would be missing!
- Agile methods [7], [11], [12] promise to deal with late changes just as they do with early changes.

There is merit in each of these claims, and there are many more approaches. However, the challenge in practice is to slice a new method so that it will be accepted and can be adopted by a business unit. In our case, the business unit saw little benefit in changing their principal process: it served them well when applied to series software development. We needed a convincing case if we wanted to suggest any drastic changes – a small subset of a method had to break the ice.

Agile methods have provoked a controversy between “classical” and “agile” opinions. Only recently did some authors exhibit a merge-and-mix rather than a either-or attitude: Boehm [13] suggests an excellent bigger picture that covers agile, classical, and risk-driven concepts.

We welcome approaches like Boehm’s or Hruschka’s [14]. Although very similar to our principle ideas, Boehm addresses the adaptation of a software development process (*before it is instantiated*), whereas we are engaged in process improvement (making rather small *changes* to an *existing and running* process). Boehm’s book [13] appeared about a year after our improvement work started, so it inspired us only “in hindsight”. We

feel encouraged and supported by his idea of tailoring and blending classical software activities with agile elements.

4. Goal-driven process improvement

Our process support group often uses SPICE (ISO 15504) and derived models as a basis for analysis and process improvement.

4.1 GQM measurement for better acceptance

However, many business units prefer a more focused approach; e.g. when they already have a certain improvement area in mind, like in the case reported here. We may still use the SPICE (or CMMi) checklists as one source of analysis, but we mainly use goal-driven approaches like QIP [4] and GQM [15] (Figure 3).

- We start by analyzing and discussing the business goals of the business unit. Goals are refined into a goal tree.
- Since we want to demonstrate the impact of our improvement activities, we use GQM (goal-question-metrics [15]) as a goal-driven measurement approach. It is widely known both in literature, and in practice.
- In GQM, business goals are further detailed and broken down to indicators and metrics that allow to assess the impact of any activity started.
- On the way, so-called abstraction sheets explicitly capture hypotheses about the current situation, and expectations on how a given improvement activity will actually change this situation.

Success of software process improvement depends on acceptance by both management and technical staff of a business unit. Engineering-style measurement is a probate way to win – and maintain – acceptance.

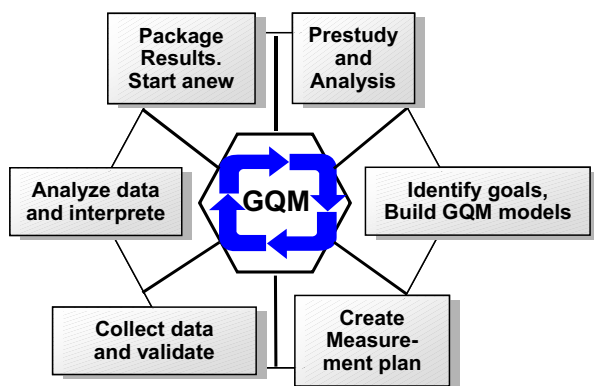


Figure 3: GQM goal-oriented steps and cyclic process

The next section summarizes (1) the results of our analysis; (2) the conclusions in terms of improvement activities proposed; and (3) their relationship to measurement. By treating agile activities just like any other, “classical” improvement activity, our approach reduces the impression of “revolution” which would diminish acceptance in a highly busy and successful software department.

4.2 Business goals

After the structural and quantitative software process analysis we determined the business goals of the bus&coach software development organization.

Increasing amounts of customer-specific software-based functions: The programmable¹ controller architecture was introduced in the year 2000. In the beginning, flashable software configurations were merely considered an instrument for lowering configuration costs in production and to reduce upgrade costs in aftersales. Additionally, they shortly grew into a feature for implementing customer-specific vehicle functions. Since it was easy to implement more complex features, the amount of software-based customer-specific functions increased drastically over the last two years. A business unit goal is to add as much value as possible to the product by offering a wide range of customer-specific options. This goal is framed by quality and economic expectations and constraints.

Zero Bug Policy: The amount of failures in the field due to problems in the software engineering process has to approach zero. The reasons are manifold: The bus company operates in the high-end market segment where customers typically do not accept malfunctions; the pressure to compensate costs of non-operational vehicles is comparatively high. Moreover, busses as public transportation systems are safety-critical products. The large amount of passengers implies a large amount of risks. Therefore, top quality is a widely accepted organizational value at all ranks.

5. Agile principles in process improvement

According to our goal-driven improvement philosophy, we worked very intensely with business unit members on different hierarchy levels. Guided by their high-level business goals (such as zero-defect), we checked “catalogues” of possible improvement activities. The catalogues included not only CMM and SPICE

¹ The controllers can be flashed via CAN in production or aftersales. The owner or driver usually cannot change the software configuration.

capability areas, but also agile approaches. Agile approaches promised improvement in terms of flexibility, speed, and high quality – exactly what we needed.

As pointed out in chapter 3, agile methods usually cannot be adopted as they stand for engineering of embedded software. Our experiences indicate, however, that agile principles can be adapted to deal with specific software process deficits in embedded development.

5.1 Software process analysis

The software process was analyzed both in structure and on a quantitative basis. In practice we started with a rather traditional and non-agile qualitative and quantitative analysis of the software process (already shown in Fig. 2).

5.2 Qualitative analysis results

1. Up to now, *not a single* failure of software functions due to engineering problems has been encountered in the field. There are several cases in which the implementation did not completely meet the customers' intentions. In those cases, the respective FUPs were redesigned and patched in a "special upload".
2. Upload Success Rate: In the first two quarters of 2003 about fifty percent of all customer-specific FUPs were uploaded in time; the rest had to be coordinated manually for later "special uploads". In former years only a few functions were not delivered in time. We attribute this perfect history to a much smaller volume of software. As pointed out above, the importance and volume of software is increasing rapidly.
3. Software affords implementing complex and useful functions at comparatively low effort. Consequently, the rate of software-based customer-specific functions increased quickly; and so did the upload failures and costly special uploads.
4. Changing customer needs or changing priorities of bus orders frequently interrupt or modify production plans. Only the sales department – and, of course, production itself – has access to the production planning system. None of the involved software departments is notified of changing upload dates.

Top quality is obvious among the highest goals of the business unit.

5.3 Quantitative Analysis Results

Rejections of FUPs at the different quality assurance check-points within the software development process are common. Our quantitative analysis brought a number of important results:

1. High internal rejection rate: 50% of all FUPs are rejected by peer reviews. Reasons include:
 - Documentation flaws, e.g. readability, wrong focus of description (high percentage of formal flaws)
 - Function coding incomprehensible.
2. Serious external rejection rate: 8% of all FUP functions reaching the system integration team (which carries out final quality assurance and release approval) are rejected. Reasons include:
 - Function unit fails specified test cases
 - Integration error
 - Function coding incomprehensible
 - Formal flaws (documentation, missing information)

In total, 58% of all functions are rejected at least once. The worst case reported were six rejection loops.

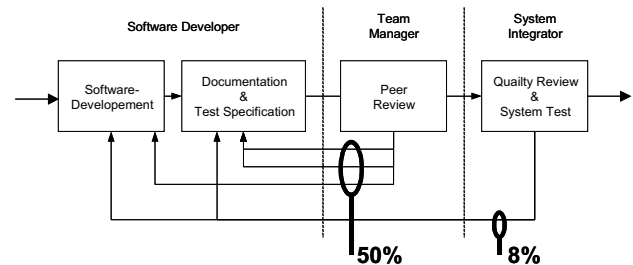


Figure 4: Rejection rates in software process

5.4 Difficulties of current work situation

We identified several recurring reasons for the above-cited problems:

- Rising work load (more software parts), but fewer development staff
- More frequent late changes (continuing to increase)
- Top quality goal (to be maintained during late changes)
- Failures leading to late arrival at upload, with resulting coordination overhead

In the next section we describe how we tried to improve the process by applying basic principles of agile practices.

6. Application of Agile Practices

We focused on four principles that are promoted by agile methods. Their roots can be tracked back to the above-cited approaches (Section 3); it is a contribution of agile methods to rephrase and combine valuable insights that have been made before by others.

6.1 Agile Practices and Improvement Activities

Based on our analysis results, we picked principles P1 through P4 and derived concrete improvement Activities A1 to A6. Additional (classical) activities were derived, but are beyond the scope of this paper.

P1: *Implement the item with the highest initial value first.*

The highest initial value was more timely software uploads. The application of this principle resulted in two improvement activities:

- A1:** All process participants get direct access to a sorted list of software functions. The sorting criterion is the upload date; the list is dynamic and reflects changes in production schedules.
- A2:** The criterion for prioritizing FUPs in development and system integration is not first-in-first-out, but Urgent First with respect to the current planned upload time.

The other principles we turned into an action were:

- P2:** *Identify discrepancies of Work Products to Specifications as automated as possible*
- P3:** *Identify discrepancies of Work Products to Specifications as soon as possible*
- P4:** *Fix discrepancies of Work Products to Specifications as soon as possible*

As pointed out above, programmers do not only write FUP software, but they also specify and document it. We considered automated Unit Test a straight-forward instantiation of P2; a Test First software process would be ideal to meet P3. And P4 is an organizational issue. To further support P2-P4, the software process was restructured to enforce early avoidance of errors:

- A3:** Start implementation by writing the test plan. When changes are necessary modify the plan in advance.
- A4:** Keep coding until the automated unit test succeeds.
- A5:** Only function that have passed developer Unit Tests are processed by the system integration department.

A6: Eliminate the extra team manager review.

In addition to the improvement activities described here, we proposed a number of classical improvements (e.g. strengthening documentation by writing less but focusing on rationale rather than description; improving the requirements elicitation process). Research and Technology was mainly supporting the introduction of agile practices.

6.2 Technical challenges

Although straight-forward in concept, a successful introduction of Test First required intense technical and organizational preparations. Without external support (in this case from Research and Technology), a busy unit can hardly spend this start-up effort. What looks so easy in XP books [7] boils down to stiff technical challenges in a practical embedded software environment like ours:

- In the past, FUP code was checked manually against semi-formal specifications by using a hardware configuration for simulation. The system integration staff could often distinguish and fix some obvious typos from real errors on the fly.
- Automated testing introduced an unintended degree of accuracy with respect to syntax and timing.
- The “test cases” were rather *specifications* written for humans to read, not for computers to execute: For example, specifying entire ranges (e.g. [1..252]) as input was common place; however, this easily results in a combinatorial explosion. Before automated processing the plan, a human expert pruned the combinatorial space by selecting critical cases from an analysis of the function. Now it became necessary to teach the developers how to write more selective specifications.

The imagination of a Test First software process rather challenges developers’ mindsets: It is still very unusual for the developers to turn from the sequence implement-document-test to test-implement-document.

7. GQM-based process improvement with agile elements

Based on the agile principles and derived activities, we hoped to increase flexibility, while preserving high quality. The goal-oriented process had led us to pick Unit Test and Test First as our starting subset.

Since goal-driven process improvement should be complemented with measurement, we needed to express agile promises in a more concrete and specific way. We derived a number of expectations or high potential

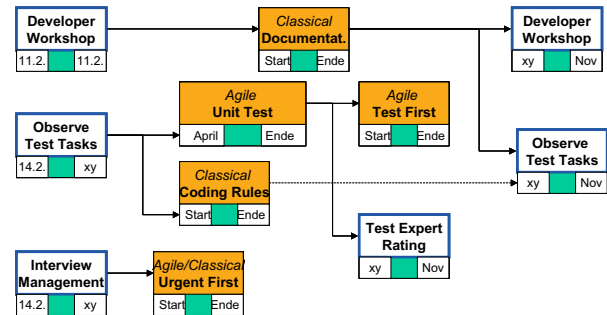
benefits that we associate with unit test in the particular environment:

- Instead of manually setting up a simulation run, developers should be enabled to run a full test suite within seconds.
- Faster module tests at the developers' desks would encourage them to do quality assurance earlier, and more frequently.
- We expect to drastically reduce the number of modules that do not comply with their specification. Each of them will then be checked by the developers before they are forwarded to the system integration group.
- Quality assurance effort would be reduced, both for developers and for the system, integration department.
- In the future, they might start describing tests before coding (Test First).
- Test First should help developers to focus on real requirements earlier.

We identified only minor risks of introducing Unit Tests since risk mitigation is easy: this change can be undone at any point. The old process will be preserved and just gain a new process branch for special customer requests.

What is interesting about our application is its role within a "blended" classical-agile software process improvement effort. It is the process in which the (regression) Unit Tests is used and the way we built and inserted the technology to support it into the running process which makes the subtle step from applying an existing test technique to improving an overall process in a meaningful way.

From the point of view of test technology and research, Unit Test is nothing new. It resembles ordinary "regression test". When it is finally executed, it technically *is an instance* of regression test. It is not the purpose of this paper to investigate or to describe new testing techniques, but to include a given testing practice in a different process environment in order to foster more agile and fast high-quality production. Figure 5 shows how Unit Test and Test First were linked into a measurement program that was specifically designed using GQM: it refers to the expectations (for Unit Test: see above), which are again broken down into questions and metrics. Measurement data is collected in a highly tailored way during the "white" activities of Figure 5. Often, there is a pre-improvement measurement, then an improvement activity, and a controlling past-improvement measurement block.



**Figure 5: Integrating improvement activities (gray) with measurement (white boxes).
(Extract from measurement plan overview)**

We started pre-measurement a few months ago, and will be able to verify (or falsify!) our expectations when we will see past-measurement. We are currently spreading Unit Test as an activity in the business unit. The complex Unit Test tool is indispensable.

Even pre-measurement brought a number of surprises when comparing the actual situation to the assumptions collected during GQM abstraction sheets. A selection of relevant results was reported in Section 5.

This paper highlights the concept of framing agile additions in a traditional measurement framework. We consider this decision an important break-through for agile acceptance. We highly recommend this hybrid approach!

8. Conclusions

Agile method authors often refer to administrative software projects. There are only a few indications of their application to embedded software [16].

However, embedded software in automotive industry is one of many application areas that could use more flexibility at higher speed and good quality. There are sufficient reasons to try "breaking the ice":

- There are many projects that could be beneficiaries, but no agile method can be applied as-is, due to specific constraints. Our case shows one practical example.
- Compared to many of the technologies currently used in embedded software development, agile approaches are still in their infancy. The community does not know for sure how they will affect embedded projects. Therefore, many hesitate to "revolutionize" too much at a time.
- We have described the challenges our introduction of Unit Test faced, and how we have resolved them. Automation without changing the actual description language was one of the important prerequisites. From our communications with many other

(embedded software) companies we know that the challenges identified are wide-spread; so our solutions should be applicable elsewhere.

- A slow step-by-step introduction should be integrated with a traditional measurement framework. This does not only foster acceptance of management and engineering staff (as we reported above), but it will also provide a sounder basis for judging agile merits (to be reported when available).
- For the purpose of this paper, we decided not to wait for “final” measurement results, but to encourage others to try a similar approach. We were able to report our experiences on a methodological level; data will be an interesting future addition.
- The embedded software community needs reliable data soon, and from many sources. Our experience at this point tells us: Measurement can (and should) be applied to “agile elements in an embedded software process improvement”!

We have seen that making expected benefits of agile elements *explicit* helps engineering staff and management to accept them as serious options rather than esoteric claims. Measurement of their *actual* impact on work practices will *sure* increase our knowledge of agile practices.

If expectations are met, measurements-observed step-by-step introduction of agile principles will greatly improve agile reputation among embedded software projects. In the end, it may actually break the ice for (more) agile development of embedded software!

References

1. Gantner, T. and K. Schneider. *Zwei Anwendungen von GQM: Ähnlich, aber doch nicht gleich*. in *MetriKon*. 2003. Ulm, Germany.
2. Paulk, M.C., et al., *The Capability Maturity Model: Guidelines for Improving the Software Process*. 1 ed. SEI Series in Software Engineering, ed. M.C. Paulk, et al. Vol. 1. 1994, Reading, Massachusetts: Addison Wesley Longman, Inc. 441.
3. CMU-SEI, *Software Acquisition Capability Maturity Model (SA-CMM)*, . 1996, Carnegie Mellon University, Software Engineering Institute.
4. Basili, V. and G. Caldiera, *Improve software quality by using knowledge and experience*. 1995, Fall: Sloan Management Review. 55-64.
5. Schneider, K. *Experience-based Training and Learning as a Basis for Continuous SPI*. in *European SEPG*. 2001. Amsterdam.
6. Schneider, K. *Active Probes: Synergy in Experience-Based Process Improvement*. in *Product Focused Software Process Improvement (PROFES 2000)*. 2000. Oulo, Finland: Springer.
7. Beck, K., *Extreme Programming Explained*. 2000: Addison-Wesley.
8. Boehm, B.W., *A Spiral Model of Software Development and Enhancement*. IEEE Computer, 1988(May 1988).
9. Lichter, H.; Züllighoven, H. *Prototyping in Industrial Software Projects - bridging the gap between theory and practice*. in *Int. Conference on Software Engineering (ICSE-15)*. 1993: IEEE Computer Society Press.
10. Schneider, K. *Prototypes as Assets, not Toys. Why and How to Extract Knowledge from Prototypes*. in *18th International Conference on Software Engineering (ICSE-18)*. 1996. Berlin, Germany.
11. Cockburn, A., *Agile Software Development*. 2002: Addison Wesley.
12. Ambler, S.W., *Agile Modeling*. 2002, Chichester: John Wiley.
13. Boehm, B. and R. Turner, *Balancing Agility and Discipline - A Guide for the Perplexed*. 2003: Addison-Wesely.
14. Hruschka, P. and C. Rupp, *Agile Softwareentwicklung für Embedded Real-Time Systems mit der UML*. 2002, München, Wien: Hanser.
15. Basili, V., G. Caldiera, and H. Rombach, *Goal question metric paradigm*, in *Encyclopedia of Software Engineering*, J.J. Marciniak, Editor. 1994, John Wiley & Sons: New York. p. 528-532.
16. Martin Lippert, S.R., Henning Wolf, *Software entwickeln mit Extreme Programming. Erfahrungen aus der Praxis*. 2002: Dpunkt-Verlag.